

Ex. No: 10a

Experiment Title: Best Fit

Aim:

To implement Best Fit memory allocation technique using python.

Procedure:

1. Input memory blocks and processes with sizes
2. Initialize all memory blocks as free.
3. Start by picking each process and find the minimum block size that can be assigned to the current process.
4. If found then assign it to the current process.
5. If not found then leave that process and keep checking the further processes.

Program:

Filename: bestfit.py

```
class Block:
    def __init__(self, size, num):
        self.size = size
        self.num = num
        self.allocated = False

class Process:
    def __init__(self, size, num):
        self.size = size
        self.num = num
        self.allocated = None
def best_fit(blocks, processes):
    print("Process No.\tProcess Size\tBlock No.")
    for p in processes:
        best = None
```

```

        for b in blocks:
            if not b.allocated and b.size >= p.size:
                if best is None or b.size < best.size:
                    best = b
        if best:
            best.allocated = True
            p.allocated = best.num
            print(f"{p.num}\t\t{p.size}\t\t{p.allocated if p.allocated
else 'Not Allocated'}")

blocks = [Block(s, i) for i, s in enumerate([100, 500, 200, 300, 600],
1)]
processes = [Process(s, i) for i, s in enumerate([212, 417, 112, 426],
1)]

best_fit(blocks, processes)

```

Sample Input and Output:

Process No.	Process Size	Block No.
1	212	4
2	417	2
3	112	3
4	426	5

Result:

Best Fit memory allocation technique using python has been successfully executed.

Ex. No: 10b

Experiment Title: First Fit

Aim:

To write a C program for implementation memory allocation methods for fixed partition using first fit.

Procedure:

1. Define the max as 25.
2. Declare the variable frag[max],b[max],f[max],i,j,nb,nf,temp, highest=0, bf[max],ff[max].
3. Get the number of blocks,files,size of the blocks using for loop.
4. In for loop check bf[j]!=1, if so temp=b[j]-f[i].
5. Check highest.

Program:

Filename: Firstfit.c

```
#include <stdio.h>

#define MAX 10

void firstFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[MAX];

    for (int i = 0; i < n; i++)
        allocation[i] = -1;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (blockSize[j] >= processSize[i]) {
                allocation[i] = j;
                blockSize[j] -= processSize[i];
                break;
            }
        }
    }
}
```

```

        }
    }
}

printf("\nProcess No.\tProcess Size\tBlock No.\n");
for (int i = 0; i < n; i++) {
    printf("%d\t\t%d\t\t", i + 1, processSize[i]);
    if (allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
}

```

```

int main() {
    int blockSize[MAX], processSize[MAX];
    int m, n;
    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
    printf("Enter sizes of memory blocks:\n");
    for (int i = 0; i < m; i++) {
        printf("Block %d: ", i + 1);
    }
}

```

```
        scanf("%d", &blockSize[i]);
    }

    printf("\nEnter number of processes: ");
    scanf("%d", &n);
    printf("Enter sizes of processes:\n");
    for (int i = 0; i < n; i++) {
        printf("Process %d: ", i + 1);
        scanf("%d", &processSize[i]);
    }

    firstFit(blockSize, m, processSize, n);
    return 0;
}
```

□

Sample Input and Output:

□ Enter number of memory blocks: 5

Enter sizes of memory blocks:

Block 1: 100

Block 2: 500

Block 3: 200

Block 4: 300

Block 5: 600

Enter number of processes: 4

Enter sizes of processes:

Process 1: 212

Process 2: 417

Process 3: 112

Process 4: 426

Process No.	Process Size	Block No.
1	212	2
2	417	5
3	112	3
4	426	Not Allocated



Result:

First Fit memory allocation technique using c has been successfully executed.